Scanfor Red — Metric Tracking, Storage & Offloading Reference

Where every tracked metric lives, how it's written, how fast it grows, and how it's offloaded to avoid database overload.

Reflects the codebase as of the current SQLite/Phase 4 architecture (backend/database/models.py, backend/services/prom_ingestor.py).

UPDATE **Five of the six recommendations below (§11, items 1–5) have been implemented and verified.**

Indexes on the four hot-path columns, the ticket auto-copy fix, the known-issue match-precedence fix, a DB-size/row-count health check, and a working retention/offload job (`backend/cron_cleanup.py` , 90-day default window) are all live. Item 6 (Postgres migration) remains a future trigger condition only — see §11.

Contents

1. Storage Overview

Every persisted metric lives in **one SQLite database file** — there is no separate metrics store, time-series DB, or message queue. The engine is defined in `backend/database/db.py`:

Engine	SQLite, accessed via SQLAlchemy ORM
File type	`.db` (SQLite binary file)
Default location	`backend/database/scanfor_red.db`
Override	env var `SCANFOR_DB_PATH` (Docker default: `/data/scanfor_red.db`, on a named volume)
Tables	7 — created idempotently on startup via `Base.metadata.create_all()`

The 7 tables, all defined in `backend/database/models.py`:

Table	Model class	Tracks
`alert_batches`	AlertBatch	One row per ingestion run (a "batch")
`alert_events`	AlertEvent	One row per alert/metric per batch — the core data
`known_issues`	KnownIssue	The known-issue catalog engineers maintain
`alert_notes`	AlertNote	Full history of triage notes per alert
`issue_status_history`	IssueStatusHistory	Audit trail of every status/category change
`prom_snapshots`	PromSnapshot	One row per ingestion run's file-comparison metadata
`prom_snapshot_files`	PromSnapshotFile	One row per individual `.prom` file read in a snapshot

Upstream of the database, the **only external file inputs** are the `.prom` text files (Prometheus exposition format) at the path configured by `SCANFOR_PROM_FILE_PATH`. These are read-only — the app never writes back to them.

2. Alert Events — the core metric record SQLITE TABLE: ALERT_EVENTS

What it is

One row is written **per parsed `.prom` metric line, on every single ingestion run** — not just when something changes. If 80 metrics are active and ingestion runs every 10 minutes, that's 80 new rows every 10 minutes, indefinitely. Written in `backend/services/prom_ingestor.py` (`_build_alert_event`), read via `GET /api/alerts` (`backend/routes/alerts.py`).

Field	Meaning
`alert_id`	Deterministic SHA-1-derived ID, unique per snapshot+alert (indexed)
`batch_id`	FK to the ingestion run that produced this row (not indexed — see §10)
`snapshot_id`	FK-like string to `prom_snapshots.snapshot_id` (not indexed — see §10)
`category` / `status`	`new` | `known` | `worsening` | `resolved`
`count`	Cumulative error count read straight from the `.prom` file (not a delta)
`previous_count`	The `count` value from the matching alert in the prior snapshot
`growth`	`count − previous_count`, computed at ingest time
`fingerprint_exact` / `fingerprint_general`	Match keys used to link this row to its predecessor across snapshots (see §8)
`first_seen` / `last_seen`	ISO timestamps; `first_seen` is carried forward from the earliest matching row
`severity`, `owner`, `known_issue_id`, `runbook_link`, `ticket_link`	Populated from the matched Known Issue, or manually via the API
`notes`	Latest note text only (full history lives in `alert_notes` — see §4)
Raw `.prom` fields	`hostname`, `raw_filename`, `log_file`, `error_type`, `error_index`, `color`, `system`, `tenant`, `system_type`, `caused_by`, `raw_known_error`, `raw_note` — copied verbatim from the source file for traceability

Why this table is the overload risk

Every table below either grows once per *batch* (slow) or once per user action (slow). `alert_events` grows once per *metric line, per batch* — the multiplier that matters most for sizing. See §9 for the math.

3. Alert Batches SQLITE TABLE: ALERT_BATCHES

One row per ingestion run (cron fire or manual "Process .prom Folder Now" click), regardless of how many files/metrics it contained. Written in `process_prom_file()`, read via `GET /api/alert-batches/latest`.

Field	Meaning
`batch_id`	Format `PROM-YYYYMMDD-HHMMSS-<6-char folder hash>`; unique, indexed
`received_time` / `received_time_display`	Parsed from the `# Generated:` header line inside the `.prom` files
`processed_at`	Wall-clock time the ingest actually ran
`total_issues_detected`	Count of alert events created in this batch
`source`, `environment`, `sender`, `email_subject`	Static/descriptive metadata (no real email ingestion yet — these are placeholder-style fields carried over from the original email-alert design)

4. Notes SQLITE TABLES: ALERT_EVENTS.NOTES + ALERT_NOTES

Two-part storage, by design

Notes are **not** a single field — they're tracked in two places simultaneously so both "latest state" and "full history" are cheap to query:

- `alert_events.notes` — a single text column holding only the *most recent* note; overwritten on every new note.
- `alert_notes` table — append-only; one row per note ever submitted, never overwritten or deleted.

Field (alert_notes)	Meaning
`alert_event_id`	FK to `alert_events.alert_id` (not indexed — see §10)
`note`	Free-text triage note
`created_by`	Defaults to `"user"` — no authenticated identity yet (Phase 3+ item)
`created_at` / `updated_at`	Timestamps

Written by: `POST /api/alerts/{alert_id}/notes` in `backend/routes/alerts.py`. **Growth:** one row per manual note submission — human-paced, not batch-paced. Low volume; not an overload risk on its own.

5. Tickets SQLITE TABLE: ALERT_EVENTS.TICKET_LINK + KNOWN_ISSUES.TICKET_LINK

There is **no dedicated tickets table** — a ticket is just a URL/ID string stored on two possible rows:

- `alert_events.ticket_link` — per-alert ticket, set via `PATCH /api/alerts/{alert_id}/ticket`
- `known_issues.ticket_link` — per-known-issue ticket, set when creating/updating a Known Issue (`POST` / `PUT /api/known-issues`)

When an alert is linked to a Known Issue via "mark known" (§6), the Known Issue's `ticket_link` is auto-copied onto the alert event whenever the alert doesn't already have one of its own (`event.ticket_link = event.ticket_link or known_issue.ticket_link`), alongside the existing `owner` and `runbook_link` copy-over.

Growth

Ticket links are set once per alert/issue by a human — negligible row growth (it's an UPDATE on an existing row, not a new row). No offloading concern.

IMPLEMENTED **Ticket auto-copy on mark-known**

Previously, a freshly-linked alert could silently show no ticket even though its parent Known Issue had one on file. Fixed in `mark_alert_known()` (`backend/routes/alerts.py`) — verified live against the running API.

6. Marked Known / Known Issues SQLITE TABLE: KNOWN_ISSUES

Two ways an alert becomes "known"

1. **Automatic** — every ingestion run matches each alert's fingerprint against the active Known Issue catalog (`find_matching_known_issue` in `backend/services/classifier.py`). A match sets `category = known` or `worsening` with no human action.
2. **Manual** — `POST /api/alerts/{alert_id}/mark-known` (`backend/routes/alerts.py`) either links the alert to an existing `known_issue_id` or creates a brand-new `KnownIssue` row on the fly, then re-classifies the alert and force-sets its category.

Field (known_issues)	Meaning
`known_issue_id`	Format `KI-NNN` , auto-incremented; unique, indexed
`fingerprint`	Optional exact/wildcard pattern matched against the alert's fingerprint string first, before falling back to scope matching
`host_scope` / `log_scope` / `error_type`	Wildcard-capable scope fields used when no fingerprint is set
`normal_count_min/max` , `normal_growth_min/max`	Thresholds — exceeding `normal_count_max` with positive growth reclassifies the alert as `worsening`
`status`	`active` | `monitoring` | `archived` — archived issues are excluded from matching but the row is never deleted
`cause` , `impact` , `resolution_steps` , `runbook_link` , `ticket_link` , `last_reviewed`	Human-maintained triage reference content

Growth: one row per distinct known issue an engineer catalogs — low volume, grows in the tens/hundreds over the life of the project, not per-ingestion. Archiving (§9) sets a status flag; it does not remove the row.

IMPLEMENTED **Scope-match specificity precedence**

Previously, `find_matching_known_issue()` (`backend/services/classifier.py`) returned the *first* matching Known Issue in catalog order, so classification could silently depend on insertion order when two active Known Issues both matched (e.g. a broad wildcard scope and a narrower one). Fixed: exact fingerprint match still always wins; among multiple scope-wildcard matches, the one with the least-wildcarded (most specific) `host_scope` / `log_scope` now wins. Verified with a test case — a `cars-*` catalog entry and a `cars-cars1-*` entry both matching the same alert, with the more specific one correctly selected.

7. Status History (audit trail) SQLITE TABLE: ISSUE_STATUS_HISTORY

Every status change — manual (`PATCH /api/alerts/{alert_id}/status`) or via mark-known — appends one row here, capturing `old_status` , `new_status` , `changed_by` , and an optional `change_reason` . This is the only place a full audit trail of triage decisions is kept; `alert_events` itself only ever shows current state.

Growth

One row per human status change — human-paced, low volume. Not an overload risk on its own, but see §9 — it cascade-deletes if its parent `alert_events` row is ever deleted, which matters for any future purge job.

8. Prometheus Snapshots & Snapshot Files — the comparison mechanism

SQLITE TABLES: PROM_SNAPSHOTS + PROM_SNAPSHOT_FILES

This is the machinery that makes growth/resolved detection possible. Two tables, one per ingestion run:

Table	Granularity	Key fields
`prom_snapshots`	One row per ingestion run	`snapshot_id`, `file_hash` (SHA-256 of all source files combined), `total_files`, `total_metrics`, `status` (`processed` / `error`)
`prom_snapshot_files`	One row per individual `.prom` file within a run	`filename`, `file_hash` (per-file SHA-256), `size_bytes`, `metric_count`, `generated_time`, `state_file` — indexed on `snapshot_id`

How "comparison" actually works

Step-by-step, from `process_prom_file()` in `backend/services/prom_ingestor.py`:

1. Combined SHA-256 hash of every source `.prom` file's bytes is computed (`folder_hash`). If it matches the last *successful* snapshot's hash, the run exits immediately — nothing is written. This is the idempotency guard.
2. Otherwise, the most recent successful `PromSnapshot` is loaded, and every `alert_events` row from that snapshot that is still *active* (`category != resolved`) becomes the comparison set (`_previous_active_alerts`).
3. Each newly parsed alert is matched against that set by `fingerprint_exact` first, then `fingerprint_general` (`_find_previous_alert`) — exact match wins so a change in `caused_by` line number doesn't silently merge into a different issue.
4. `growth = new count − matched previous count`; unmatched alerts are brand new (`previous_count = 0`).
5. Any previously-active alert with **no** match in the new snapshot is emitted as a synthetic `resolved` row (`_build_resolved_event`) — this only fires once per disappearance, not on every subsequent run, because resolved alerts are excluded from the next comparison set.

Comparison always looks back exactly one snapshot

There is no multi-snapshot trend/rolling-window comparison — only "this run vs. the single most recent successful run." Historical `prom_snapshots` rows before that are pure archival record; nothing in the app queries them for comparison once a newer snapshot exists.

9. Growth Rate & Volume Projection

Assuming the documented cron/watcher cadence of **every 10 minutes** (144 runs/day):

Table	Rows written per run	Rows/day (144 runs)	Notes
alert_batches	1	144	Fixed, regardless of metric volume
prom_snapshots	1	144	Fixed
prom_snapshot_files	1 per source `.prom` file (H)	144 × H	H = number of monitored hosts
alert_events	1 per active metric line (M) + resolved rows (R)	**144 × (M + R)**	The dominant term — written in full every run, not incrementally
alert_notes	0–few (human-paced)	negligible	Not batch-driven
issue_status_history	0–few (human-paced)	negligible	Not batch-driven
known_issues	~0 (catalog, human-paced)	negligible	Bounded, low volume

Worked example — plug in real production numbers for H and M once known:

```
H (hosts/files)     = 20
M (active metrics)  = 100

alert_events/day    = 144 × 100        =  14,400 rows/day
alert_events/year   = 14,400 × 365     ≈  5,256,000 rows/year
prom_snapshot_files  = 144 × 20        =   2,880 rows/day  ≈  1,051,200 rows/year
alert_batches + prom_snapshots         =     288 rows/day  ≈    105,120 rows/year combined
```

SQLite handles millions of rows fine *for reads with proper indexes*, but at this scale two things bite: (a) unindexed hot-path columns turn into full-table scans as the table grows (§10), and (b) the `.db` file itself grows unbounded on disk since nothing ever deletes old rows (§11).

10. Current Offloading Status

IMPLEMENTED **A retention/offload job now runs on a daily cadence, separate from the 10-minute ingest cycle.**

`backend/cron_cleanup.py` (invoked via `python3 -m backend.cron_cleanup`, intended for a daily cron entry) exports and deletes `alert_events` rows — plus their `alert_notes` and `issue_status_history` children — older than `SCANFOR_RETENTION_DAYS` (default 90). A row is only ever offloaded if it also has no open ticket and isn't tied to an active (non-archived) Known Issue, and the single most recent successful snapshot is always protected regardless of age, since `prom_ingestor.py` depends on it for the next growth comparison (§8). Every deleted row is written to CSV in `SCANFOR_EXPORT_DIR` first. Orphaned `alert_batches` / `prom_snapshots` / `prom_snapshot_files` (zero remaining `alert_events`) are cleaned up in the same pass, and `VACUUM` runs at the end to actually reclaim disk space. A manual trigger is also available: `POST /api/retention/run`, `GET /api/retention/status`.

Tested against a throwaway copy of the production-shaped local database with all rows backdated 200 days: 620 of 737 `alert_events` rows were correctly offloaded, 117 were correctly protected, 3 orphaned batches/snapshots and 54 orphaned snapshot files were cleaned up, exports were written with matching row counts, the `known_issues` catalog was untouched, and the `.db` file shrank from 892 KB to 282 KB after `VACUUM`.

Related scaling risk: missing indexes on hot-path columns — fixed

SQLAlchemy only indexes a column when `index=True` is explicitly set. These frequently-filtered columns were previously **not** indexed:

Column	Used by	Risk as table grows
`alert_events.snapshot_id`	`_previous_active_alerts()` — runs on *every ingestion*	Full table scan every 10 minutes
`alert_events.batch_id`	`GET /api/alerts` — runs on *every dashboard load*	Full table scan on every page load/refresh
`alert_notes.alert_event_id`	Note history lookups	Scan grows with total notes ever written
`issue_status_history.alert_event_id`	Audit trail lookups	Scan grows with total status changes ever made

IMPLEMENTED **`index=True` added to all four columns**

Since this project has no migration tool (no Alembic — confirmed, not in `requirements.txt`), `Base.metadata.create_all()` alone would not have retroactively indexed these columns on an already-existing database. Added `ensure_indexes()` in `backend/database/db.py` — idempotent `CREATE INDEX IF NOT EXISTS` DDL run at startup by both the web server (`main.py`) and the standalone ingest/cleanup scripts. Verified against the existing local `.db` file: all four indexes are present without a reset.

11. Prioritized Recommendations

Items 1–5 below have been implemented and verified (marked IMPLEMENTED). Item 6 is still a future trigger condition only, not something built.

NOW Cheap, safe, no data-loss risk

IMPLEMENTED **1. Add the missing indexes**

Effort: small · Risk: none · Unlocks: faster ingestion + faster dashboard loads immediately

`index=True` added in `backend/database/models.py` to `alert_events.snapshot_id`, `alert_events.batch_id`, `alert_notes.alert_event_id`, and `issue_status_history.alert_event_id`.

Gotcha handled: this project has no migration tool

No Alembic (confirmed — not in `requirements.txt`, no `alembic/` directory), so `Base.metadata.create_all()` alone would not have retroactively indexed these columns on a database that already existed. Added `ensure_indexes()` (idempotent `CREATE INDEX IF NOT EXISTS` DDL) in `backend/database/db.py`, run at startup by both the web server and the standalone ingest/cleanup scripts — verified against the existing local `.db` without needing a reset.

IMPLEMENTED **2. Fix the ticket_link and Known-Issue precedence gaps**

Effort: small · Risk: none · See §5 and §6 callouts above for detail

`known_issue.ticket_link` now auto-copies onto a newly-linked alert when the alert has none of its own; explicit specificity precedence now applies when multiple active Known Issues could match the same fingerprint.

IMPLEMENTED **3. Add a DB-size / row-count health check**

Effort: small · Risk: none · Gives visibility before overload becomes an incident

`GET /api/health` now reports `alert_events_row_count` and `db_size_bytes` alongside the existing status fields — verified live (`alert_events_row_count: 737, db_size_bytes: 892928` against the local dev DB).

NEXT Requires a design decision, moderate effort

IMPLEMENTED **4. Retention/offload job, parallel to `cron_ingest.py`**

Effort: medium · Risk: low, export-before-delete · Retention window: 90 days (owner-approved)

`backend/cron_cleanup.py` , meant for a daily cron entry (not every 10 min):

1. Selects `alert_events` rows older than `SCANFOR_RETENTION_DAYS` (default 90) AND not tied to an active Known Issue (`known_issue_id IS NULL` or the linked issue is archived) AND with no open ticket (`ticket_link IS NULL`)
2. Exports the selected rows to CSV before deleting (see item 5)
3. Bulk-deletes matching `alert_notes` and `issue_status_history` rows first, then the `alert_events` rows themselves
4. Deletes orphaned `alert_batches` / `prom_snapshots` / `prom_snapshot_files` rows once no `alert_events` reference them — **except** the single most recent successful snapshot, always kept since §8 depends on it for the next comparison
5. Runs `VACUUM` at the end, on a separate connection outside the retention transaction (SQLite can't `VACUUM` inside an open transaction)

A manual trigger also exists: `POST /api/retention/run` , `GET /api/retention/status` .

Verified against a throwaway DB copy

All 737 local `alert_events` rows were backdated 200 days on a copy of the database (never the real one) and the job run against it: 620 rows correctly offloaded, 117 correctly protected, 3 orphaned batches + 3 orphaned snapshots + 54 orphaned snapshot files cleaned up, exports written with matching row counts, `known_issues` left untouched, and the file shrank from 892 KB to 282 KB after `VACUUM` . A live run against the real (untouched) local DB correctly returned `no-op` , since nothing there is actually 90+ days old yet.

IMPLEMENTED **5. Export-before-delete to cold storage**

Effort: medium · Risk: none, purely additive · Pairs with item 4

CSV chosen over Parquet — no new dependency required (`csv` is stdlib) versus Parquet's `pyarrow` / `pandas` addition. Exports land in `SCANFOR_EXPORT_DIR` (default: an `exports/` folder next to the database file, so a Docker deployment's exports live on the same persisted volume as the DB) as three dated files per run — `*_alert_events.csv` , `*_alert_notes.csv` , `*_issue_status_history.csv` .

LATER Only if volume actually outgrows SQLite

6. Migrate history tables to Postgres

Effort: large · Risk: medium · Trigger condition, not a default plan

SQLite's single-writer lock is the real ceiling here — cron writing every 10 minutes while the dashboard reads concurrently is already flagged as a risk in the project's own docs (`database is locked` troubleshooting entry, Appendix A). If the `.db` file reaches multi-GB size, or lock contention starts showing up in logs, that's the trigger to move `alert_events` / `prom_snapshots` history to Postgres — already anticipated in the Phase 3 docs. Don't do this preemptively; it's a real infrastructure lift for a problem the current scale likely doesn't have yet.

12. Quick Reference Table

Metric	Storage	File type	Written by	Growth pace	Offloading today
Alert events (count/growth)	`alert_events` table	SQLite row	`prom_ingestor.py`, every ingest	Fast — per metric, per run	Yes — `cron_cleanup.py`, 90-day window, export-before-delete (§11)
Alert batches	`alert_batches` table	SQLite row	`prom_ingestor.py`, every ingest	Fixed, per run	Yes — deleted once orphaned (zero remaining alert_events), latest always protected
Notes	`alert_events.notes` + `alert_notes` table	SQLite row(s)	POST /alerts/{id}/notes	Human-paced	Yes — offloaded alongside their parent alert_events row
Tickets	`alert_events.ticket_link`, `known_issues.ticket_link`	SQLite column (URL/ID string)	PATCH /alerts/{id}/ticket, known-issue CRUD	Human-paced (UPDATE, not INSERT)	N/A — no growth; a row with an open ticket is also exempt from retention (§11)
Marked known / Known Issues	`known_issues` table	SQLite row	POST /alerts/{id}/mark-known, known-issue CRUD	Slow — catalog-sized	Status flag only (archived), never deleted or touched by retention
Status change history	`issue_status_history` table	SQLite row	PATCH /alerts/{id}/status, mark-known	Human-paced	Yes — offloaded alongside their parent alert_events row
Snapshots (comparison basis)	`prom_snapshots` table	SQLite row	`prom_ingestor.py`, every ingest	Fixed, per run	Yes, once orphaned — but the single latest snapshot is never deleted, since comparison always needs it
Snapshot files (per-host detail)	`prom_snapshot_files` table	SQLite row	`prom_ingestor.py`, every ingest	Fast — per host, per run	Yes — deleted along with their orphaned parent snapshot
Raw source metrics	filesystem	`.prom` text files	External Scanfor tool (not this app)	Overwritten in place, not appended	N/A — read-only, not owned by this app
Database file itself	filesystem / Docker volume	`.db` (SQLite)	SQLAlchemy engine	Bounded by the 90-day retention window instead of growing forever	Yes — `VACUUM` runs at the end of every `cron_cleanup.py` run

Generated from the current backend source (models.py, prom_ingestor.py, routes/*.py, classifier.py). Re-generate after any schema or ingestion-logic change.